

Master's thesis
Master's Programme in Computer Science

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Ha Vien

August 7, 2026

Supervisor(s): Keijo Heljanko, Christoph Luneburg, Petteri Kotimaa

Examiner(s):

University of Helsinki
Faculty of Science

P. O. Box 68 (Pietari Kalmin katu 5)
00014 University of Helsinki

HELSINGIN YLIOPISTO — HELSINGFORS UNIVERSITET — UNIVERSITY OF HELSINKI

Tiedekunta — Fakultet — Faculty	Koulutusohjelma — Utbildningsprogram — Degree programme
Faculty of Science	Master's Programme in Computer Science

Tekijä — Författare — Author

Ha Vien

Työn nimi — Arbetets titel — Title

Performance, Safety, and Engineering Trade-offs of Rust and C++ in Nokia RAN Control-Plane Systems

Työn laji — Arbetets art — Level	Aika — Datum — Month and year	Sivumäärä — Sidantal — Number of pages
Master's thesis	August 7, 2026	21

Tiivistelmä — Referat — Abstract

Abstract to be written last.

ACM Computing Classification System (CCS):
Software and its engineering → Software notations and tools → General programming languages
Computer systems organization → Real-time systems → Real-time system specification
Networks → Network types → Mobile networks

Avainsanat — Nyckelord — Keywords

Rust, C++, RAN control plane, tail latency, memory safety, fault injection, FFI

Säilytyspaikka — Förvaringsställe — Where deposited

Helsinki University Library

Muita tietoja — Övriga uppgifter — Additional information

Software study track

Contents

1 Introduction **1**

2 Background: Systems Languages for Latency-Critical Software **3**

3 Problem Context: UE Context Management and Predictability **5**

4 Objectives **7**

5 Research Methodology **9**

6 Design and Implementation **11**

7 Experimental Setup and Evaluation **13**

8 Discussion **15**

9 Conclusion and Future Work **17**

Bibliography **19**

Appendix A Fault-injection example **21**

1. Introduction

- The 5G RAN control plane (gNB-CU-CP) is being softwarized onto commodity servers: it is large, stateful, highly concurrent, and bound by millisecond-scale procedure budgets [2], so a defect or a tail-latency regression there degrades the whole site.
- That software is written predominantly in C++, which supplies the required control and performance but leaves memory-safety and concurrency defects to the programmer [13]; Rust promises the same performance with those defects rejected at compile time [10].
- The problem: telecom vendors face growing pressure to adopt memory-safe languages [3] but have no direct, controlled evidence on predictability, prevented-defect scope, or migration cost for a representative control-plane subsystem [9].
- Four research questions frame the work: RQ1 tail latency and jitter under overload [4], RQ2 which defect classes are eliminated at compile time, RQ3 the overhead of Rust's safety model, and RQ4 the cost of incremental adoption inside a C++ codebase.
- Contributions: a reproducible benchmarking framework, a frozen specification with two parity-tested implementations, a defect-class taxonomy including the boundary of the guarantee, and evidence-based migration guidelines.

2. Background: Systems Languages for Latency-Critical Software

- The NG-RAN architecture splits the gNB into central and distributed units and the central unit again into control and user planes; UE Context Management sits inside the gNB-CU-CP, connected over the standardised F1, E1, and NG interfaces [1].
- Predictability rather than average speed decides whether deadlines are met, so the governing metrics are the p99/p99.9 tail and jitter [4], measured past saturation and captured without coordinated omission using high-dynamic-range histograms [14].
- C++ offers zero-overhead abstractions, RAII, and no garbage collector, but safety remains the programmer's responsibility [13] and sanitizers detect only what a given test happens to execute.
- Rust enforces memory safety and data-race freedom at compile time through ownership, borrowing, and the `Send`/`Sync` traits [8], with `unsafe` as an explicitly delimited escape hatch and no garbage collector.
- The comparison is therefore C++ against Rust (managed languages are excluded on tail-latency grounds), with the concurrency model held fixed and C-ABI interoperability as the adoption path; no prior study isolates the language this way [9].

3. Problem Context: UE Context Management and Predictability

- UE Context Management owns the per-UE context lifecycle: a four-state machine (Idle, Connected, Reconfiguring, Released) driven by a small, lifecycle-oriented operation set in which every illegal operation returns a typed error.
- It is chosen because it is both representative (per-UE mutable state, strict lifecycle, high concurrency, hard latency targets [2]) and tractable enough to specify precisely and parity-test in two languages.
- Five defect classes lie inside Rust's guarantee [8] and each has a natural incarnation here: use-after-free, double-free, iterator invalidation, data race, and unguarded shared mutation.
- Two further classes are carried deliberately from outside the guarantee, a lock-order deadlock and a lost update, so that the study locates the boundary instead of confirming a foregone conclusion.
- Fan-out and fleet scale turn the p99.9 of a single operation into a sustained rate of degraded procedures [4], which is why the load sweep is pushed over the overload cliff.

4. Objectives

- A stated literature-search protocol returns no study measuring the tail latency of a Rust and a C++ implementation of the same stateful control-plane subsystem under overload [9].
- Five objectives map onto the research questions: quantify predictability (O1), determine prevented defect classes (O2), measure resource overhead (O3), build and measure a migration prototype (O4), and deliver reusable artifacts and guidance (O5).
- Three hypotheses are committed to in advance: H1 performance equivalence below saturation [5], H2 compile-time prevention of the defect classes C++ leaves to best-effort detection, and H3 negligible interoperability cost.
- Success means decision-grade evidence: repeats with confidence intervals [7], defects demonstrated on compilable programs, migration cost read off a working prototype, and reproducibility from published artifacts.
- Scope is drawn to protect internal validity; the full gNB, PHY/L2 and the user plane, captured traffic traces, managed languages, and production operations are explicitly out.

5. Research Methodology

- The approach is empirical and constructive: rather than comparing two pre-existing systems, the study builds a matched pair of implementations from one frozen specification so that language is the only systematic difference.
- RQ1 and RQ3 are answered by an open-loop load sweep from light load into overload, with latency captured in identically configured HDR histograms [14] and resource use sampled through the operating system.
- RQ2 is answered by controlled fault injection [12]: a fixed catalogue is confronted in both languages, with recorded expectations in each direction so an unexpected outcome surfaces as a discrepancy.
- RQ4 is answered by a working C++-to-Rust FFI prototype measuring seam size, the scope of `unsafe`, and per-call crossing cost [11].
- Fairness is the central experimental control (frozen specification, matched allocator and optimisation, forced-idiom log), and results are reported as medians with bootstrap confidence intervals plus exact paired permutation tests under Holm correction [7]; all data is synthetic, so no data-protection concerns arise.

6. Design and Implementation

- A language-agnostic specification is frozen before any code is written, fixing the public API, state machine, data structures, algorithmic strategy, and sharding and concurrency model as the fairness baseline.
- Both versions use one shared-nothing, thread-per-shard architecture: eight shards, bounded blocking MPSC queues, single-use completion handles, strict FIFO per shard, and a single atomic counter as the only shared mutable state.
- The run-time substrate is equalised: the same jemalloc allocator (with a system-allocator control), identically configured HDR histograms, matched `-O2` and `opt-level = 2`, and pinned shard-selection and intra-shard hash functions.
- Where the languages must diverge, the forced-idiom log records it: C++ hand-writes the bounded queue Rust gets from `sync_channel`, and C++'s tagged request struct plus response variant is larger and more branch-heavy than Rust's per-variant typed sender, leaving Rust longer in lines but lower in total cyclomatic complexity [15].
- The Rust core contains no `unsafe` at all, and functional equivalence is tested rather than assumed by diffing canonical transcripts from a language-neutral conformance suite.

7. Experimental Setup and Evaluation

- Two environments are distinguished: everything reported was measured on a WSL2 development host, which is adequate for parity, fault injection, and the FFI structure but not for tail-latency claims; the bare-metal protocol is specified and scripted but not yet run.
- The workload is a deterministic, spec-justified RAN lifecycle generator driven on a fixed open-loop send schedule and swept from nominal load through saturation, with the four language-by-allocator variants interleaved within every repeat.
- RQ1 and RQ3 performance: the four variants are indistinguishable at the median and equal in throughput and CPU [5], while the tail is not resolvable here because the confidence interval on one variant's p99 exceeds the spread between all four.
- RQ3 memory is the one cleanly resolved performance result: the Rust build carries a few MiB more peak resident set under either allocator, with the same sign in every repeat and a larger effect than the allocator's own.
- RQ2 and RQ4: all five in-scope defect classes are compile errors in safe Rust [8] while each compiles in C++ and needs the right sanitiser, the two boundary classes compile in both, and the twelve-entry-point FFI seam keeps the core `unsafe`-free at a sub-nanosecond per-call cost.

8. Discussion

- The findings are reported with an evidence status per research question: safety and migration are final because they do not depend on the measurement environment, while performance and resource results are preliminary.
- On performance the honest reading is that the median question is settled and the tail question is open, which keeps the eventual bare-metal result a finding rather than a confirmation; the memory penalty is bounded but unexplained without hardware counters.
- On safety the difference is one of kind rather than degree, and it has a definite edge: Rust eliminates data races, not race conditions, and aliasing errors, not lock-ordering errors [8].
- On migration, an `unsafe`-free core is a statement about what the compiler checks and not about what the C++ host sharing its address space can corrupt [11], so the gain is that the component stops contributing new defects.
- The hypotheses are revisited (H1 half-answered, H2 supported precisely and refuted in its loose reading, H3 supported), engineering guidelines are derived, and related work, threats to validity, and the professional obligation not to over-claim are addressed.

9. Conclusion and Future Work

- The thesis compared modern C++ and Rust on one UE Context Management service built from a single frozen specification, isolating language effects from design and skill differences along four dimensions.
- Safety: safe Rust rejects all five injected memory-safety and data-race classes at compile time, while the two boundary classes compile and misbehave exactly as they do in C++.
- Migration: an unmodified Rust core can be driven from C++ across a thin, `unsafe`-isolated seam at sub-nanosecond per-call cost, so the cost is API design and the learning curve [6].
- Performance: the languages are equivalent at the median and in CPU and throughput, the Rust build carries a few MiB more peak RSS, and the tail is not yet measurable on a shared host.
- Future work runs the specified bare-metal campaign first, then extends to production traces, further subsystems, hardware counters, async runtimes, in-process heap isolation, and a longitudinal migration case study.

Bibliography

[1] 3GPP. NG-RAN; architecture description. Technical Specification (TS) 38.401, 3rd Generation Partnership Project (3GPP), 2022. V17.5.0, Release 17.

[2] 3GPP. Service requirements for the 5G system; stage 1. Technical Specification (TS) 22.261, 3rd Generation Partnership Project (3GPP), 2023. V18.6.0, Release 18.

[3] Cybersecurity and Infrastructure Security Agency and National Security Agency and Federal Bureau of Investigation and others. The case for memory safe roadmaps: Why both C-suite executives and technical experts need to take memory safe coding seriously. Cybersecurity information sheet, CISA, NSA, FBI, ASD's ACSC, CCCS, NCSC-UK, NCSC-NZ, CERT NZ, Dec. 2023.

[4] J. Dean and L. A. Barroso. The tail at scale. *Communications of the ACM*, 56(2):74–80, 2013.

[5] P. Emmerich, S. Ellmann, F. Bonk, A. Egger, E. García Sánchez-Torija, T. Günzel, S. Di Luzio, A. Obada, M. Stadlmeier, S. Voit, and G. Carle. The case for writing network drivers in high-level programming languages. In *Proceedings of the 2019 ACM/IEEE Symposium on Architectures for Networking and Communications Systems (ANCS)*, 2019.

[6] K. R. Fulton, A. Chan, D. Votipka, M. Hicks, and M. L. Mazurek. Benefits and drawbacks of adopting a secure programming language: Rust as a case study. In *Proceedings of the 17th Symposium on Usable Privacy and Security (SOUPS)*, pages 597–616. USENIX Association, 2021.

[7] A. Georges, D. Buytaert, and L. Eeckhout. Statistically rigorous Java performance evaluation. In *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, pages 57–76, 2007.

[8] R. Jung, J.-H. Jourdan, R. Krebbers, and D. Dreyer. Safe systems programming in Rust. *Communications of the ACM*, 64(4):144–152, 2021.

[9] R. Karlbäck and A. Orö. Holistic view on alternative programming languages for radio access network applications in cloud and embedded deployments. Master's thesis, Linköping University, Department of Computer and Information Science, 2021. Carried out at Ericsson AB.

[10] S. Klabnik and C. Nichols. *The Rust Programming Language*. No Starch Press, San Francisco, CA, 2nd edition, 2023.

[11] Z. Li, J. Wang, M. Sun, and J. C. S. Lui. Detecting cross-language memory management issues in Rust. In *Computer Security – ESORICS 2022*, pages 680–700, 2022.

[12] R. Natella, D. Cotroneo, and H. S. Madeira. Assessing dependability with software fault injection: A survey. *ACM Computing Surveys*, 48(3):44:1–44:55, 2016.

[13] L. Szekeres, M. Payer, T. Wei, and D. Song. SoK: Eternal war in memory. In *Proceedings of the 2013 IEEE Symposium on Security and Privacy (S&P)*, pages 48–62, 2013.

[14] G. Tene. HdrHistogram: A high dynamic range histogram, and the coordinated omission problem. http://hdrhistogram.org/, 2015. Accessed 2026.

[15] T. Yin. Lizard: An extensible cyclomatic complexity analyzer for many programming languages. http://www.lizard.ws/, 2025. Version 1.23.0. Accessed 2026.

Appendix A. Fault-injection example

- Illustrates one case from the RQ2 catalogue, the use-after-free, in both languages side by side.
- In C++ an alias into the context map survives the `erase` that frees it, and the program compiles cleanly.
- It is flagged only at runtime, and only by AddressSanitizer on a path a test actually exercises.
- The analogous safe-Rust program does not compile: the borrow checker forbids mutating the map while a reference into it is live (`E0499`) [10].
- The pair makes the kind-of-guarantee difference concrete rather than argued from language semantics.